

Techniques Used

Technique Used	Purpose in which Used
If/Else statements	To prevent nil values, conduct user input validation.
For loops (For Each)	To create repeating UI components from arrays and dictionaries of data.
Functions	For reusability of commonly performed actions, ability to execute a set of code when certain conditions are met.
Structs and Classes	SwiftUI stores components/screens as a structure. Classes used to model the format of persistent data.
Extensions	For reusability of commonly performed actions on existing data types, e.g. format Date object into string.
Binding variables	For flexible storage and representation of user input between frontend and backend.
Computed variables	For defining variables that are computed using another variable defined earlier.
Filtering, grouping by setting conditions	To filter arrays down to items that meet user's searched criteria, forming dictionaries to use in formation of pie charts.
SwiftData (Creating, Updating, Deleting, Querying Persistent Data)	To store user preferences and expense data on device in a persistent manner, so that it can be fetched and displayed in the UI.
Use of API	To get exchange rates between currencies for given dates, represent expenses in default currency.
Concurrency and Error Handling	To have fallback code when updating persistent storage (model context) fails, or when API request fails.

Table 3: *Table of techniques used in programming the solution, and purpose of technique.*

Justification of Code

Binding variables using @State, .onAppear to allow User Input

To allow users to create their own cards, categories, and expenses in order to meet success criteria #2 and #3, I used Forms that allow for user input, which can then be stored for instances of cards, currencies, expenses, etc.

I wanted it so that the default selection was the default Currency and Category set by the user. There was an issue where in Swift, you could not use values of a variable initialized in the struct to initialize another. Furthermore, you cannot use computed values for variables with the State property. To work around this issue, I used the .onAppear function within the SwiftUI.

```
Unset
struct newExpense: View {
    @Environment(\.modelContext) private var context // Set up a model context
    environment, from modelContext defined elsewhere in project
    @Query private var creditCards: [creditCard] // Retrieve array of
    creditCard objects stored in persistent storage
    @Query private var currencies: [Currency] // Same as above for Currency
    @Query private var categories: [Category] // Same as above for Category

    @State private var currency: Currency? // variable currency can be changed
    according to user's interaction with app UI
    @State private var card: creditCard? // "?" makes it an optional, meaning
    it can either be a creditCard value, or nil
    @State private var category: Category?
```

First, I queried all instances of the creditCard, Currency, and Category models so that I can show them in the interface for user selection. This is done by setting the modelContext environment, then using @Query to retrieve an array of all instances of the model. Then, I created variables with the State property, to represent the user's selection. The state property gives variables the ability to be read and written through user interaction with SwiftUI interfaces. By binding these variables to the outputs of components on the form, user input can be stored and handled as they are updated. Variables with the state property require a default value. However, this cannot be set to our liking due to the issue mentioned above. Hence, I make it an optional value for now.

```
Unset
var body: some View {
```

```

NavigationStack {
    Form {
        Section("Card") { // Example Section for Card selection,
similar sections repeated for each choice
            Picker("Select Card:", selection: $card){
                ForEach(creditCards, id: \.self) { card in
                    Text(card.name).tag(card as creditCard?)
                }
            }
            .pickerStyle(.menu)
        }

        .onAppear { // When this Form loads in UI
            if currency == nil { // initially nil as currency is
an initialized as an optional
                currency = currencies.first(where: {
                    $0.isDefault })} // first in user's
currencies where isDefault attribute is true
            if card == nil {
                card = creditCards.first }
            if category == nil {
                category = categories.first(where:{
                    $0.isDefault })}}
        }
    }
}

```

For the UI view, I create pickers so that the user can select from existing Credit Cards, Currencies, and Categories. The example above shows code for just the credit card picker. I use the ForEach structure to compute a selectable Text view associated with each item in the array defined above. To allow for the user selection to be reflected in the backend, I bind the State variables to the user's selection from the picker.

As of now, when the view is first loaded, variables with the State property are nil due to their optional status. Hence, I use .onAppear to perform a series of if statements that first checks whether the variables are nil, and if so, set its value as the first instance from each array where its isDefault attribute (a custom attribute created when defining the model) is set to true.

Use of extensions onto existing Swift data types

To meet success criteria #2 there was a need to create instances of cards to store in the model context, and display them in the GUI so the user could see the cards under their account. As the user will likely create multiple cards, I chose to allow the user to select different colors for UI card components. However, an issue I encountered when trying to store these instances of cards in persistent storage, was that SwiftData does not allow for direct storage of SwiftUI Color

data types. Another limitation was that the SwiftUI Color data type does not yet allow for direct conversion from hexadecimal representation, but can retrieve a color by passing in the red, green, blue, and opacity values. Hence, I chose to create an extension to the SwiftUI Color data type so I can directly obtain the RGB values of the desired color as an array for easy storage and fetching of the data.

```
Unset
extension Color {
    func asRGB() -> [Double] { // creates method to SwiftUI Color type
        let color = UIColor(self).CGColor // represent SwiftUI Color as UIKit
        UIColor type, allowing use of the existing method CGColor
        let RGB = color.components // split CGColor into RGB components
        var output: Array<Double> = [] // define empty array
        for n in RGB! { // loop through all values
            output.append(Double(n)) // append value of red, green, blue,
            alpha to array
        }
        return output
    }
}

func extractSwiftUIColor(RGB: [Double]) -> Color {
    return Color(
        red: RGB[0],
        green: RGB[1],
        blue: RGB[2],
        opacity: RGB[3]
    )
}
```

By creating an extension, I programmed a new method for Color by defining the function `asRGB`, which returns an array of doubles. Using self to refer to the SwiftUI Color itself, I used the existing `CGColor` method to obtain the CoreGraphics representation of the same color. This is then split into an array containing floats of the RGB values using the method `components`. Considering that when converting the RGB value back into a SwiftUI Color, it expects to be passed values of the double data type, I used a for loop to convert the array of floats into an array of doubles. Hence this extension allows for the easy conversion of a SwiftUI Color into an array of double values, which can be stored in SwiftData. To reduce redundant code, I also chose to define the function `extractSwiftUIColor` to extract the SwiftUI Color from the RGB values stored in the app's persistent data.

Use of computed properties and filter

To meet success criteria #4 and #5, there was a need to filter all past expenses down into just expenses within the current month, so that further graphs and statistics can be computed from it. To reduce code redundancy I create a function to return an array of all expenses within the current month.

Unset

```
func sortThisMonth(expenses: [Expense]) -> [Expense]{
    let currentDate = Date() // initialize a Date object
    let calendar = Calendar.current // current refers to device user's current
    timezone, format of representation of date, etc.
    let currentYear = calendar.component(.year, from: currentDate) // use
    currentDate and calendar to retrieve this year
    let currentMonth = calendar.component(.month, from: currentDate) //
    retrieve this month

    let filteredExpenses = expenses.filter { expense in // iterates through
    each expense in expenses array
        let expenseYear = calendar.component(.year, from: expense.date) //
    obtain year of the expense
        let expenseMonth = calendar.component(.month, from: expense.date) //
    obtain month of the expense
        return expenseYear == currentYear && expenseMonth == currentMonth //
    add expense to filteredExpenses if the expense year and month match current
    year and month
    }

    return filteredExpenses
}
```

First I define the current date as an object of the `Date()`, which by default is initialized as the current system date and time. Next, I also define a `Calendar` object. A `Calendar` object allows for the comparison of calendar units (such as year, month, day, etc.) from a given absolute point in time. Using this calendar variable I also set the `currentYear` and `currentMonth` variables, so that I can compare the dates of expenses against them and find those that were created within the month. This is retrieved using the `component` method on the calendar to get the respective year and month components from the value of the date stored in the expense instance.

I then create a computed variable, `filteredExpenses`, to store the resulting array of expenses from the current month. To calculate the value stored in this variable, I use the `filter` method to filter through the expenses array. I specify that I want to iterate through each item (expense) in the array, from each item taking the components of year and month stored in `expense.date`. Using these values I can compare them against the `currentYear` and `currentMonth` defined earlier, returning only the expenses where both the year and month match to form the `filteredExpenses`.

Sources Referenced in Development

Apple Developer Documentation, Apple, developer.apple.com/documentation. Accessed 1 Sept. 2024.

Arai, Chino. “【swift】urlcomponentsを使ったネットワークリクエスト.” *Qiita*, 18 July 2023, qiita.com/imchino/items/615ef4baf683cfd91d3b.

Ardyan, Muhammad Rizki. “Currency.Swift.” *Gist*, GitHub, 22 Oct. 2023, gist.github.com/ardyanrizki/4c3e6b2f027666669e8f211a1586b7fe.

“Free Currency Conversion API.” *Freecurrencyapi.Com*, Everapi GmbH, freecurrencyapi.com/. Accessed 1 Nov. 2024.

Harrison, Keith. “Swiftui Pie Charts.” *Use Your Loaf - iOS Development News & Tips*, 29 Apr. 2024, useyourloaf.com/blog/swiftui-pie-charts/.

Hudson, Paul. “Filtering the Results from a Swiftdata Query.” *Hacking with Swift*, Hacking with Swift, 12 Oct. 2023, www.hackingwithswift.com/quick-start/swiftdata/filtering-the-results-from-a-swiftdata-query.

Hudson, Paul. “How to Read the Red, Green, Blue, and Alpha Color Components from a UIColor.” *Hacking with Swift*, Hacking with Swift, 28 May 2019, www.hackingwithswift.com/example-code/uicolor/how-to-read-the-red-green-blue-and-alpha-color-components-from-a-uicolor.

Hudson, Paul. “How to Save a Swiftdata Object.” *Hacking with Swift*, Hacking with Swift, 31 Oct. 2023, www.hackingwithswift.com/quick-start/swiftdata/how-to-save-a-swiftdata-object.

Ishikawa, Ryo. “【Swift】Dateの王道【日付】.” *Qiita*, 13 Aug. 2017, qiita.com/rinov/items/bff12e9ea1251e895306.

Jabrayilov, Majid. “Mastering Charts in SWIFTUI. Pie and Donut Charts.” *Swift with Majid*, 26 Sept. 2023, swiftwithmajid.com/2023/09/26/mastering-charts-in-swiftui-pie-and-donut-charts/.

Lee, Antoine van der. “What Is a Computed Property in Swift?” *SwiftLee*, 12 Jan. 2021, www.avanderlee.com/swift/computed-property/.

Nakagawa, Kensuke. “【swiftui】onappearの使い方と活用方法を徹底解説！.” *iPhoneアプリ開発講座 | CodeCandyオンラインプログラミングスクール*, 4 Apr. 2023, blog.code-candy.com/swiftui_onappear/.

Narita, Motoki. “Swiftのmap, Filter, Reduce(などなど)はこんな時に使う！.” *Qiita*, 2 Dec. 2015, qiita.com/motokiee/items/cf83b22cb34921580a52.

Pradeilles, Vincent. “How to Write Your First API Call in Swift.” *Swift with Vincent*, Swift with Vincent, 21 May 2023, www.swiftwithvincent.com/blog/how-to-write-your-first-api-call-in-swift.

Prater, Karin. “Data Handling in Swiftdata: Create, Read, Update, Delete.” *SwiftlyPlace*, 3 July 2023, www.swiftlyplace.com/blog/crud-in-swift-data.

Prater, Karin. “SWIFTUI Foreach: More than Just Loops in Swift.” *SwiftlyPlace*, 22 May 2023, www.swiftlyplace.com/blog/swiftui-foreach-more-than-just-loops-in-swift.

Shunta. “Optional(Wrapped)型を理解しよう！.” *Qiita*, 9 Oct. 2020, qiita.com/shunta01/items/f3db4706a2866bea1eaf.

Sugawara, Yu. “SwiftData 入門.” *Zenn*, 11 Nov. 2023, zenn.dev/yumemi_inc/articles/2a929c839b2000.

Suzuki, Junji. “iOS 13時代の Locale（言語と地域）基本おさらい【重要】.” *Qiita*, 29 Oct. 2019, qiita.com/Junji_Suzuki_JBOYSOFT/items/1f11f0cf3c7bee7018e6.

Tagishi, Masakatsu. “【学習備忘録】(Swift)Urlsessionで取得したデータを画面に反映させる.” *Zenn*, 13 May 2022, zenn.dev/masakatsu_tagi/articles/f5374dd3153bdc.

Uhooi. “UserDefaultsの概要と操作方法(Swift).” *Qiita*, 13 Feb. 2019, qiita.com/uhooi/items/429cac9b798b9c0937ae.

Vahter, Toomas By Toomas. “Interactive Pie Chart in SwiftUI.” *Augmented Code*, 30 Oct. 2023, augmentedcode.io/2023/10/30/interactive-pie-chart-in-swiftui/.

Vargas, Joe. “Swift’s Dictionary Grouping and How Underrated It Is.” *Medium*, CodeX, 12 Apr. 2023, medium.com/codex/swifts-dictionary-grouping-and-how-underrated-it-is-8de68a50c4f7.

Walker, Eric. “Checking for the User’s First Launch in Swift.” *Medium*, Better Programming, 28 Oct. 2019, betterprogramming.pub/checking-for-the-users-first-launch-in-swift-df02a1feb472.